DESARROLLO DE APLICACIONES II

Guion de presentación

Grupo 9 · Core

13 de octubre de 2026

Contenido

0 · Apertura — 30 s 3
1 · El hallazgo fuerte: SEC-08 — 90 s 3
2 · Los otros dos hallazgos — 60 s 3
3 · La parte incómoda: nuestros tests estaban mal — 75 s 4
4 · La verificación con Fable — 60 s 4
5 · Cierre — 30 s 5
Versión de 2 minutos 5
Si preguntan 5

Duración: ~6 minutos. Al final hay una versión de 2 minutos por si se corta el tiempo, y las preguntas más probables con su respuesta.

Lo que está en *cursiva* son indicaciones, no se dicen.

---

0 · Apertura — 30 s

Arrancá con el PDF cerrado. Que lo primero sea lo que encontraron, no el índice.

> Nosotros somos el Core, el módulo 9: el pasamanos de eventos. Todo lo que se mandan los otros ocho módulos pasa por nosotros.
>
> Armamos los diez criterios y los corrimos contra nuestro propio código. Siete pasan. **Tres no.**
>
> Y uno de los tres permite que cualquiera de ustedes, desde su cuenta normal, le corte la integración a otro equipo sin que el otro se entere.
>
> Empiezo por ese.

Pausa corta. Recién ahí abrís el PDF.

---

1 · El hallazgo fuerte: SEC-08 — 90 s

Mostrá la sección SEC-08 del PDF, la parte de la evidencia.

> El Core deja que cada equipo cargue un JSON Schema para sus tipos de evento. Ese schema es lo que usamos para validar el contenido de cada evento de ese tipo.
>
> El endpoint pide estar autenticado. Pero **no chequea que seas el dueño** del tipo de evento.

Señalá el bloque de evidencia.

> Acá nos logueamos como Rentas y reescribimos el schema de `workOrderScheduled`, que es un evento de Obras. Devolvió **200**. Nos dejó.
>
> Y esto es lo importante: no es que rompimos un campito en una tabla. Después de eso hicimos que Obras publicara su evento normal, el de todos los días.

Señalá el segundo bloque.

> **422. Rechazado. Y a la cola de errores.**
>
> O sea: con un request, desde una cuenta común, le mandás a la DLQ todo el tráfico de un tipo de evento de otro equipo. Y el otro equipo no tiene cómo enterarse, salvo que se le ocurra mirar su DLQ.
>
> El arreglo son tres líneas. Las suscripciones y las publicaciones **ya** chequean el dueño. El endpoint del schema es el único que se olvidó.

---

2 · Los otros dos hallazgos — 60 s

Pasá a la tabla de hallazgos abiertos.

> Los otros dos, más rápido.
>
> **Sin límite de tasa en el login.** Cuarenta intentos seguidos, cuarenta respuestas, ningún 429.

Pero acá hay un matiz que nos obligamos a poner, porque la primera versión estaba exagerada: nosotros **sí** tenemos bloqueo de cuenta a los cinco intentos, y eso frena la fuerza bruta contra una contraseña. Lo que no frena es dejar a los nueve equipos afuera con cinco requests cada uno, los emails son predecibles. Ni el password spraying. Ni el agotamiento de CPU.

Y encontramos algo que no habíamos visto: **una vez bloqueada la cuenta, los intentos siguientes se siguen procesando igual.** Ciento noventa milisegundos de bcrypt cada uno. El bloqueo no le ahorra trabajo al servidor.

Y el tercero: datos personales en los logs. Cuando entra un mensaje malformado, escribimos los primeros doscientos caracteres en el log. Lo probamos con un DNI inventado y ahí está, en claro. En producción eso va al agregador de logs, que lo ve mucha más gente que la base.

---

3 · La parte incómoda: nuestros tests estaban mal — 75 s

Esta es la parte que más va a valer. No la saltees.

Ahora les quiero contar algo que nos pasó, porque creo que es lo más útil de todo el taller.

La primera corrida nos dio cuatro de diez. Seis criterios fallando. Casi lo entregamos así.

Fuimos a mirar uno por uno y **tres de esos seis eran bugs nuestros, de los tests, no del sistema.**

El peor: teníamos un test que decía que Rentas estaba viendo eventos de Atención Ciudadana, y lo reportamos como fuga de datos entre módulos. No era una fuga. Rentas **está suscripto** a ese tipo de evento, así que esos eventos le fueron entregados. Verlos es exactamente lo que tiene que pasar.

El test estaba preguntando lo que no era. Para probar aislamiento había que buscar un módulo que no fuera **ni origen ni destinatario**, y eso lo tuvimos que ir a buscar en los datos.

Otro comparaba el `traceId`, que es aleatorio en cada request, así que daba distinto siempre.

La moraleja para nosotros fue esta: **un test de seguridad que da rojo no es un hallazgo todavía.** Hay que ir a ver por qué dio rojo. Si hubiéramos entregado la primera corrida, reportábamos tres vulnerabilidades que no existen.

---

4 · La verificación con Fable — 60 s

El punto cuatro de la consigna pedía verificar al menos tres criterios con Fable.

Nosotros le dimos **los tres que fallaban**, no tres que pasaran. Y le pedimos explícitamente que tratara de **refutarlos**: que si alguno no se sostenía, lo bajara.

Los tres se sostuvieron. Pero nos corrigió en tres cosas:

Uno: **nos bajó la severidad del rate limiting**, de alta a media, porque nuestra redacción daba a entender fuerza bruta ilimitada y eso era exagerado.

Dos: encontró lo del bcrypt después del bloqueo, que se nos había escapado.

Tres: en el del schema, nosotros habíamos **afirmado** que los eventos iban a ir a la DLQ. Él fue y lo **probó**: hizo publicar a Obras y capturó el rechazo.

Y eso es lo que convierte «se puede editar un campo ajeno» en «le podés cortar la integración a un equipo».

---

5 · Cierre — 30 s

Volvé a la tabla de hallazgos.

> Resumiendo: siete de diez pasan, tres no, los tres tienen ticket en Jira con etiqueta seguridad.
>
> El que arreglamos primero es el del schema. Es el único de severidad alta, es el más fácil de explotar, y el daño le cae a otro equipo.
>
> Y una cosa que les conviene a ustedes: si alguno quiere chequear su propia integración, tenemos un endpoint, `dashboard/integration-alerts`, que cruza quién publica qué con quién escucha qué. Así encontramos, sin buscarlo, que Obras espera un evento que Tránsito publica con otro nombre.

Versión de 2 minutos

Si te cortan el tiempo, decí sólo esto:

1. Somos el Core. Diez criterios, corridos contra nuestro código. **Siete pasan, tres no.**
2. **El grave:** cualquier módulo autenticado puede reescribir el JSON Schema de un evento ajeno. Lo probamos: Rentas reescribió un schema de Obras, y después los eventos legítimos de Obras empezaron a ir a la DLQ. Negación de servicio contra otro equipo, desde una cuenta común.
3. Los otros dos: sin rate limiting en el login, y datos personales que quedan en los logs.
4. **La primera corrida nos dio 4 de 10, y tres de esos fallos eran bugs de nuestros tests, no del sistema.** Un test en rojo no es un hallazgo hasta que entendés por qué está en rojo.
5. Los tres hallazgos los verificamos aparte con Fable, pidiéndole que los refutara. Se sostuvieron, y nos bajó una severidad que teníamos inflada.

Si preguntan

«¿Por qué no tienen criterios de mobile?» > La consigna lo contempla: sin app, van dos de datos sensibles. El Core es > backend más un dashboard web, no tiene app.

«¿Por qué sólo dos de integración y no SOAP?» > Porque la integración entre los nueve módulos es por RabbitMQ, no por SOAP. > Los dos criterios son sobre el contrato del sobre y sobre quién puede tocar > los schemas, que es lo que define esa integración.

«¿Lo corrieron contra producción?» > No hay producción todavía, no está desplegado. Corrimos contra desarrollo, > sobre una copia descartable de la base, porque uno de los tests bloquea una > cuenta y otro reescribe un schema.

«¿Ya lo arreglaron?» > Todavía no, están los tres tickets abiertos. El del schema es el próximo: > son tres líneas, pasar el módulo que llama y compararlo con el dueño.

«¿Y si el schema lo tiene que poder editar el admin?» > Sí, y eso queda: la regla sería dueño **o** admin. Hoy no hay ninguna de las dos.

«¿Cómo sabés que los siete que pasan pasan de verdad?» > Pusimos controles positivos. En los de permisos no alcanza con ver un 403: > también verificamos que el admin sí pueda. Si no, un servicio que responde 403 > a todo pasaría el test.